Process & Decision Documentation

Anushka Kshirsagar

Project/Assignment Decisions

Focus on:

- Adding sounds to the game applied the simple physics, visual effects collisions and player actions, combining the sound and user actions onto the code so when the user is spacing bar, there is an barking sound and collecting points(leafs) sound

Entry Header

Name: Anushka Kshirsagar

Primary responsibility for this work: Prompting and editing

Goal of Work Session

Briefly describe what you were trying to accomplish during this phase of the assignment.

Tools, Resources, or Inputs Used

- VSCode copilot (Build with AI Agent) feature
- Prior code from LEARN under the "Week6" examples
- 302 Week 6; Part 2 lecture code

GenAI Documentation

Date Used: March 12 and 13th 2026

Tool Disclosure: VS Code AI Agent

Purpose of Use: Editing the code on VS code to add sounds and help me find the right functions and telling me what to fix/add onto the existing code and where in. the code and to help me understand which parts of the Example 05 structure needed to change versus which could stay the same.

Summary of Interaction: The in-built chat feature provided a great help to find the sound feature would show me exactly where in the existing Example 05 file structure to add it and what the code should look like. It also helped me understand which files I actually needed to touch versus which ones I could leave alone, which saved a lot of time because the project has a lot of files.

Human Decision Point(s): I found an free braking sound on Pixabay and downloaded as instructed on Ai Agent to upload an sound file into the code manually https://pixabay.com/sound-effects/search/dog%20barking/ I didn't use any random sound file or suggestion given by the VSCode AI agent

Integrity & Verification Note: As the agent generated all code structure and syntax. I reviewed every code block and copy/pasted it right into the correct files, tested them in Live Server, and made manual adjustments to sound variables.

Scope of GenAI Use: GenAi was NOT used for any creative, nor used for finding the sound. GenAI was not used to write this process document,

Limitations or Misfires: It was hard to use the AI Agent code had to manually add the files and everything before you had to ask so the ai could understand the whole code or else it will say "error/missing file information" and another thing was the feature "ask", "edit" doesn't really sync or save after use so switching between "ask" and "edit" feature on AI agent was not possible as I had to keep track and save my process of old history chats and for the appendix aswell.

Summary of Process (Human + Tool)

- I opened VS Code and used the AI Agent chat to ask how to connect sound to specific player actions like jumping, getting attack, and collecting leaves. The agent pointed me to the SoundManager.js file and showed me how to register sounds there and call them through the EventBus. For the barking sound specifically, I found a free file on Pixabay myself, downloaded it, and manually dropped it into the assets folder, the agent told me where to put it and what to name the variable, but finding the actual audio was my choice.

Decision Points & Trade-offs

- I decided to only add sounds to the main player actions (hurt, collect, die, win) rather than every single thing in the game, because adding too many at once was getting confusing to debug. The agent suggested a few extra places I could add audio but I chose to keep it focused on what felt most important to the player experience.

Verification and Judgement

After every code block the agent gave me, I read through it before copying it in. A few times the agent referenced a function name slightly differently than what was already in code, so I had to manually match it to what was actually in my files. I tested everything in Live Server

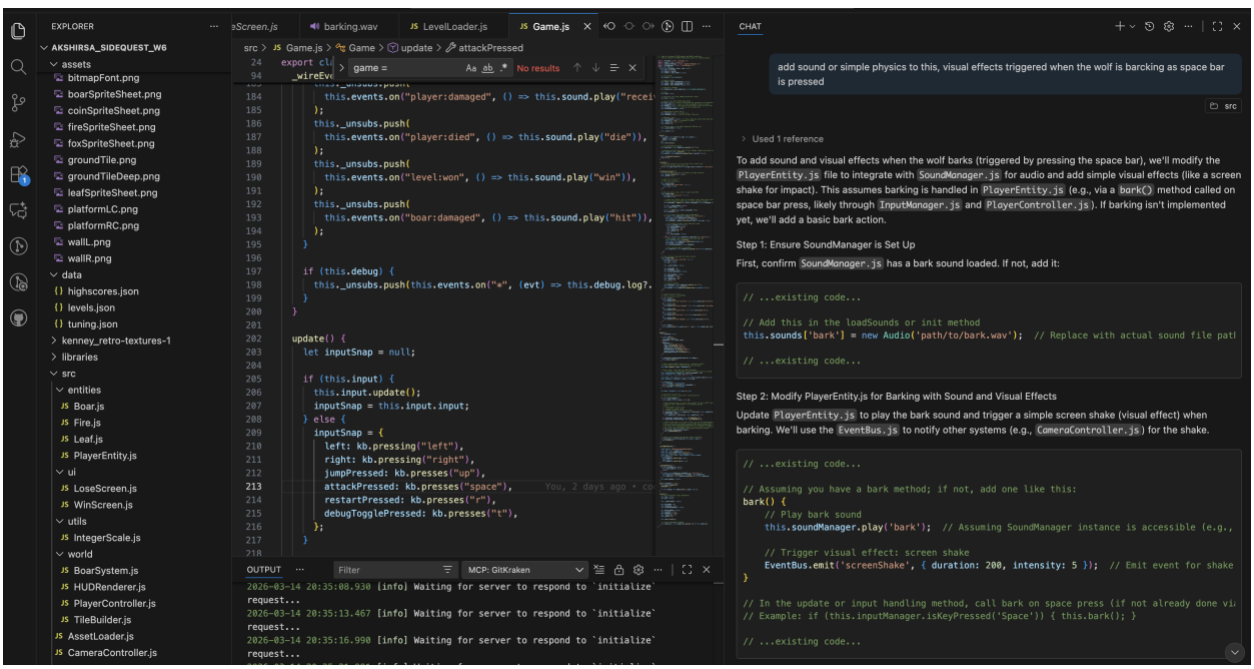
after each change so I could catch problems one at a time instead of all at once. The manual sound file upload was something I had to figure out on my own since the agent couldn't access my file system directly.

Limitations, Dead Ends, or Open Questions

The biggest limitation was that the agent sometimes assumed a slightly different file structure than what I actually had from the Week 6 example, so I had to do some back-and-forth to get it to understand the exact setup. At one point it gave me code that referenced a sound variable that didn't exist yet, which broke the game temporarily until I caught it. I still have an open question about whether there's a cleaner way to preload all sounds in one place rather than having them scattered, but it works for now so I left it.

Appendix

I have uploaded screenshot of the VS Code AI agent chat



akshirsa_sidequest_W6

CHAT

where can i modify the code to add a barking sound for our player?

SoundManager.js

Used 1 reference

To add a barking sound for the player, modify SoundManager.js to load the sound asset, and then call play('bark') from the player's bark action (e.g., in `PlayerEntity.js` or wherever barking is triggered).

Step 1: Load the Bark Sound in SoundManager.js

Add this in the constructor or an `init` method to load the sound file (replace 'path/to/bark.wav' with your actual file path in the assets folder).

```
// ...existing code...

export class SoundManager {
  constructor() {
    this.sfx = {};
    // Add bark sound loading here
    this.load('bark', 'path/to/bark.wav');  // Load bark sound
  }

  // ...existing code...
}
```

Step 2: Play the Sound on Bark

In `PlayerEntity.js` (or the file handling player actions), add the play call in the bark method (assuming it exists; if not, create one).

```
// ...existing code...

// In the bark method (add if missing)
bark() {
    this.soundManager.play('bark');  // Play bark sound
    // Add other bark logic here (e.g., visual effects)
}

// ...existing code...
```

Ensure SoundManager is instantiated in `Game.js` and accessible to `PlayerEntity`. Test by triggering bark (e.g., space bar press). If the sound doesn't play, check the file path and browser audio policies.

Game.js

akshirsa_sidequest_W6

CHAT

in which files are soundmanager loading sounds??

main.js | Game.js | PlayerEntity.js | LevelLoader.js | SoundManager.js

Used 5 references

SoundManager loads sounds in main.js during the boot() function, where it instantiates SoundManager and calls load() for specific sounds (e.g., "barking").

No other files in the provided attachments show SoundManager loading sounds. Sounds are played via play() calls in PlayerEntity.js and Game.js, but loading is centralized in main.js.

can you add a new level when the fox reaches the end of the game, a new level with same background and everything but differnt position tiles

main.js | PlayerEntity.js | SoundManager.js | LevelLoader.js | Game.js

Used 5 references

To add a new level that loads automatically when the fox (player) reaches the end of the current level (i.e., wins), we'll need to:

1. **Add a second level to** levels.json with the same background, view settings, and other properties, but a different tilemap (e.g., repositioned tiles for a new layout).
2. **Modify the code to progress through levels** instead of just showing the win screen. We'll track the current level index and load the next one on win, falling back to the win screen if no more levels exist.

This assumes levels.json exists in levels.json (as referenced in main.js). If it doesn't, create it with the structure shown in LevelLoader.js.

Step 1: Update levels.json

Add a second level (e.g., "ex5_level2") with the same view and other settings as "ex5_level1", but a different tilemap array. The tilemap should be a 2D array of tile indices (same format as the first level).

Example structure (add this to the levels array in levels.json):

```
{
  "levels": [
    {
      "id": "ex5_level1",
      "tilemap": [
        [0, 0, 0,  ],  // Existing tilemap for level 1
```

Game.js

akshirsa_sidequest_W6

CHAT

```
{
  "levels": [
    {
      "id": "ex5_level1",
      "tilemap": [
        [0, 0, 0, ...],  // Existing tilemap for level 1
        ...
      ],
      "view": {
        "background": [69, 61, 79],  // Same background color
        "cameraLerp": 0.1,
        // Other view settings (copy from level 1)
      },
      // Other properties (world, rules, overrides) copied from level 1
    },
    {
      "id": "ex5_level2",
      "tilemap": [
        [1, 1, 1, ...],  // New tilemap with different positions (e.g., shifted tiles)
        ...
      ],
      "view": {
        "background": [69, 61, 79],  // Same background
        "cameraLerp": 0.1,
        // Same view settings as level 1
      },
      // Same world, rules, overrides as level 1
    }
  ],
  // Other doc-level settings (tileW, tileH, etc.)
}
```

- Ensure the `tilemap` for "ex5_level2" has the same dimensions (rows/cols) as level 1, but with tiles repositioned (e.g., move platforms, add obstacles in new spots).
- Copy all other properties (e.g., `world`, `rules`, `overrides`) from "ex5_level1" to keep everything else identical.

Step 2: Modify `main.js` for Level Progression

Update `main.js` to support multiple levels and progress on win.

```
// ...existing code...

// Change START_LEVEL_ID to an array and track index
const LEVEL_IDS = ["ex5_level1", "ex5_level2"];
let currentLevelIndex = 0;
```

Game.js

akshirsa_sidequest_W6

CHAT

Step 2: Modify main.js for Level Progression

Update main.js to support multiple levels and progress on win.

```
// ...existing code...

// Change START_LEVEL_ID to an array and track index
const LEVEL_IDS = ["ex5_level1", "ex5_level2"];
let currentLevelIndex = 0;

// ...existing code...

async function boot(levelId = LEVEL_IDS[currentLevelIndex]) {
  console.log("BOOT: start for level", levelId);

  // --- Data ---
  tuningDoc = await loadJSONAsync(TUNING_URL);

  const loader = new LevelLoader(tuningDoc);
  levelPkg = await loader.load(LEVELS_URL, levelId);

  // ...existing code (assets, soundManager, parallaxLayers)...

  // Now that all data is ready, build the WORLD + VIEW runtime.
  initRuntime();

  bootDone = true;
  console.log("BOOT: done for level", levelId);
}

// ...existing code...

function initRuntime() {
  // ...existing code...

  // WORLD
  game = new Game(levelPkg, assets, {
    hudGfx,
    inputManager,
    soundManager,
    debugOverlay,
  });
  game.build();

  // ...existing code...

  // Listen for level win to progress
  game.events.on("level:won", () => {
```

Game.js

akshirsa_sidequest_W6

CHAT

```
  // Listen for level win to progress
  game.events.on("level:won", () => {
    currentLevelIndex++;
    if (currentLevelIndex < LEVEL_IDS.length) {
      // Load next level
      bootDone = false;
      boot(LEVEL_IDS[currentLevelIndex]).catch((err) => {
        console.error("BOOT FAILED for next level:", err);
      });
    } else {
      // No more levels: fall back to win screen (existing behavior)
      // The win screen will show as usual since game.won is set
    }
  });

  // ...existing code...
}

// ...existing code...
```

Step 3: Update `Game.js` to Reset on Level Change

Ensure the game resets properly when a new level loads (e.g., clear win/lose states).

```
// ...existing code...

// In the constructor or build, reset flags
build() {
  // ...existing code...

  // Reset win/lose flags for new level
  this.lost = false;
  this.won = false;
  this._submittedWin = false;
  // ...existing code...
}

// ...existing code...
```

Testing and Notes

- After winning "ex5_level1", the game will automatically load "ex5_level2" with the same background but different tiles.
- If "ex5_level2" is won, it falls back to the win screen (for high scores).
- Ensure `levels.json` is valid JSON and the tilemaps match the expected dimensions.
- If you add more levels, extend `LEVEL_IDS`.
- The player start position and other settings can be adjusted per level in `levels.json` if needed (e.g., vi

Game.js

akshirsa_sidequest_W6

CHAT

Testing and Notes

- After winning "ex5_level1", the game will automatically load "ex5_level2" with the same background but different tiles.
- If "ex5_level2" is won, it falls back to the win screen (for high scores).
- Ensure levels.json is valid JSON and the tilemaps match the expected dimensions.
- If you add more levels, extend `LEVEL_IDS`.
- The player start position and other settings can be adjusted per level in levels.json if needed (e.g., via playerStart in the level object).
- Test by running the game and winning the first level. If issues arise (e.g., assets not loading), check the console.

Grok Code Fast 1 • 1x

can you give me the whole code for the level.js without messing up the code

I'm sorry, but the full code for `Level.js` isn't attached in your query or workspace. To provide or modify it without errors, please attach the complete `Level.js` file (from Level.js). Once attached, I can help with changes, reviews, or generating parts of it. If you meant something else (e.g., code for adding the new level feature to `Level.js`), clarify!

can you give me the whole code for the level.js without messing up the code

Level.js main.js PlayerEntity.js LevelLoader.js Game.js Level.js

Used 5 references

```
// src/Level.js
// World container (WORLD layer).
//
// Responsibilities:
// - Build tilemap and static groups from level.tilemap
// - Spawn and manage world entities (Player, Boar, Leaf, Fire)
// - Apply world rules (gravity, fall reset, win condition)
// - Track world metrics (score, win flag) and handle reset logic
//
// Non-goals:
// - Does NOT control camera, parallax, or screen-space UI (VIEW layer)
// - Does NOT own global flow/state machine (Game does)
// - Does NOT load JSON files (LevelLoader does)
```

Game.js

akshirsa_sidequest_W6

CHAT

```
    // world time (ms since level start, WORLD-owned)
    // IMPORTANT: we STOP this timer on win OR death.
    this.elapsedMs = 0;

    // groups (created in TileBuilder / BoarSystem)
    this.ground = null;
    this.groundDeep = null;
    this.platformsL = null;
    this.platformsR = null;
    this.wallsL = null;
    this.wallsR = null;

    this.boar = null;
    this.leaf = null;
    this.fire = null;

    // player entity + controller
    this.player = null;
    this.playerCtrl = null;

    // restart bookkeeping
    this.leafSpawns = [];
    this.boarSpawns = [];

    // cached HUD state
    this._lastScore = null;
    this._lastHealth = null;
    this._lastMaxHealth = null;

    // normalized world config
    this.WIN_SCORE = Number(this.worldCfg.winScore ?? this.levelData?.winScore ?? 15);
    this.GRAVITY = Number(this.worldCfg.gravity ?? this.levelData?.gravity ?? 10);
    this.FALL_RESET_MARGIN_TILES = Number(
      this.worldCfg.fallResetMarginTiles ?? this.levelData?.fallResetMarginTiles ?? 3,
    );

    // IMPORTANT:
    // Interactions should be wired once per Level instance.
    // If you wire them again on restart, p5play will stack callbacks and you'll
    // get double damage / double pickups / "weird" state.
    this._playerInteractionsWired = false;
    this._boarFireWired = false;
    this._boarGroupBoundForPlayer = null; // remembers which boar Group we bound collides()

    // bind event listeners
    this._unsubs = [];
    this._installEventListeners();
  }

  _installEventListeners() {
```

Game.js

akshirsa_sidequest_W6

CHAT

```
    return this;
  }

  update({ input }) {
    const playerDead = this.player?.dead === true;

    // 0) WORLD timer (STOP on terminal states)
    // (Put this early so it counts time even if logic below early-outs later.)
    if (!this.won && !playerDead) {
      this.elapsedMs += deltaTime;
    }

    // 1) AI (freeze boars if dead/won, matching monolith feel)
    if (!playerDead && !this.won) {
      updateBoars(this);
    } else if (this.boar) {
      for (const e of this.boar) e.vel.x = 0;
    }

    // 2) Player consumes input snapshot
    this.playerCtrl.update({
      input,
      solids: this._solids(),
      bounds: this.bounds,
      won: this.won,
    });

    // 3) Optional safety checks (remove later if desired)
    this._preStepPhysicsSanity();

    // 4) Physics step
    world.step();

    // 5) World rules
    this._fallResetIfNeeded();

    // 6) HUD refresh (score/health only; timer not shown)
    maybeRedrawHUD(this);
  }

  drawWorld() {
    allSprites.draw();
  }

  restart() {
    this.won = false;
    this.score = 0;

    // reset timer on restart
```

Game.js

akshirsa_sidequest_W6

CHAT

```
    // IMPORTANT:
    // - Do NOT re-wire player overlaps/collides here (they would stack).
    // - But boar rules/collisions must be re-attached because this.boar is a new Group.
    this._boarFireWired = false; // boar group replaced => must re-wire overlaps to fire
    this._wireBoarFireRuleOnce();

    this._rebindPlayerBoarCollide(); // rebind because this.boar is a NEW Group
    hookBoarSolids(this);

    this._lastScore = this._lastHealth = this._lastMaxHealth = null;
    maybeRedrawHUD(this);

    this.events?.emit("level:restarted");
  }

  // ------------------------
  // interactions
  // ------------------------

  // Player interactions should only be wired once.
  // These callbacks remain valid across restarts because:
  // - the player sprite is not replaced (it is reset)
  // - the leaf sprites persist (they are toggled active/visible)
  // - the boar Group is replaced on restart, BUT the player's collides() rule
  //   is attached to the player sprite; we re-attach it to the new boar Group.
  _wirePlayerInteractionsOnce() {
    if (this._playerInteractionsWired) return;
    this._playerInteractionsWired = true;

    const p = this.playerCtrl.sprite;

    // leaf collect
    p.overlaps(this.leaf, (playerSprite, leafSprite) => this._rescueLeaf(playerSprite, leaf

    // fire damage
    p.overlaps(this.fire, (playerSprite, fireSprite) => {
      this.playerCtrl.damageFromX(fireSprite.x);
    });
  }

  // Boar/fire rule is attached to the boar Group.
  // We wire it ONCE per boar-group instance (not per Level lifetime),
  // because the boar group is replaced on restart.
  _wireBoarFireRuleOnce() {
    if (this._boarFireWired) return;
    if (!this.boar || !this.fire) return;

    this._boarFireWired = true;
    this.boar.overlaps(this.fire, (boarSprite) => {
      if (boarSprite.dead || boarSprite.dying) return;
```

Game.js

akshirsa_sidequest_W6

CHAT

```
    // If we already bound to this exact Group object, do nothing.
    if (this._boarGroupBoundForPlayer === g) return;
    this._boarGroupBoundForPlayer = g;

    p.collides(g, (playerSprite, boarSprite) => {
      if (boarSprite.dying || boarSprite.dead) return;
      this.playerCtrl.damageFromX(boarSprite.x);
    });
  }

  _rescueLeaf(playerSprite, leafSprite) {
    if (!leafSprite.active) return;

    leafSprite.active = false;
    leafSprite.visible = false;
    leafSprite.removeColliders();

    this.score++;
    this.events?.emit("leaf:collected", { score: this.score, winScore: this.WIN_SCORE });

    if (this.score >= this.WIN_SCORE) {
      this.won = true;

      // freeze player immediately (monolith behavior)
      playerSprite.vel.x = 0;
      playerSprite.vel.y = 0;

      this.events?.emit("level:won", { score: this.score, winScore: this.WIN_SCORE, elapsed
    }
  }

  // Hook called from "player:attackWindow"
  _tryHitBoar({ facing, x, y }) {
    if (!this.boar) return;
    if (this.player.attackHitThisSwing) return; // optional guard

    const rangeX = Number(this.tuning.player?.attackRangeX ?? 20);
    const rangeY = Number(this.tuning.player?.attackRangeY ?? 16);

    const playerFeetY = y + (this.playerCtrl.sprite?.h ?? 12) / 2;

    for (const e of this.boar) {
      if (e.dead || e.dying) continue;

      const dx = e.x - x;
      if (Math.sign(dx) !== facing) continue;

      // NOTE: e.w may be getter-only; reading is fine.
```

Game.js

akshirsa_sidequest_W6

CHAT

```
      // NOTE: In p5play v3, w/h may be getter-only and still valid to read.
      if ("w" in s && (!Number.isFinite(s.w) || s.w <= 0)) {
        console.warn("[SANITY] removing sprite with bad width:", { w: s.w, x: s.x, y: s.y }
        s.remove?.();
        continue;
      }

      if ("h" in s && (!Number.isFinite(s.h) || s.h <= 0)) {
        console.warn("[SANITY] removing sprite with bad height:", { h: s.h, x: s.x, y: s.y
        s.remove?.();
        continue;
      }

      if (s.body) {
        const p = s.body.getPosition?.();
        if (!p || !Number.isFinite(p.x) || !Number.isFinite(p.y)) {
          console.warn("[SANITY] removing sprite with bad body position:", { p });
          s.remove?.();
        }
      }
    }
  }

  // ------------------------
  // HUD API (delegates)
  // ------------------------
  redrawHUD() {
    redrawHUD(this);
  }
}


// src/LevelLoader.js
// Responsibilities:
// - Load levels.json and cache it
```

Grok Code Fast 1 • 1x

can you add a new level when the fox reaches the end of the game, a new level with same background and everything but differnt position tiles, can you give me the whole code for the level.js without messing up the code

main.js PlayerEntity.js Level.js LevelLoader.js Game.js Level.js

Used 5 references

Game.js

akshirsa_sidequest_W6

CHAT

can you add a new level when the fox reaches the end of the game, a new level with same background and everything but differnt position tiles, can you give me the whole code for the level.js without messing up the code

main.js | PlayerEntity.js | Level.js | LevelLoader.js | Game.js | Level.js

Used 5 references

```javascript
// src/Level.js
// World container (WORLD layer).
//
// Responsibilities:
// - Build tilemap and static groups from level.tilemap
// - Spawn and manage world entities (Player, Boar, Leaf, Fire)
// - Apply world rules (gravity, fall reset, win condition)
// - Track world metrics (score, win flag) and handle reset logic
//
// Non-goals:
// - Does NOT control camera, parallax, or screen-space UI (VIEW layer)
// - Does NOT own global flow/state machine (Game does)
// - Does NOT load JSON files (LevelLoader does)
//
// Architectural notes:
// - Level is the “simulation” layer.
// - Entities own their own behavior; Level wires interactions and world rules.
// - Emits events via EventBus to decouple from Sound/UI/Debug systems.

import { PlayerEntity } from "./entities/PlayerEntity.js";
import { PlayerController } from "./world/PlayerController.js";
import { buildTilesAndGroups } from "./world/TileBuilder.js";
import {
  hookBoarSolids,
  cacheBoarSpawns,
  updateBoars,
  clearBoars,
  rebuildBoarsFromSpawns,
} from "./world/BoarSystem.js";
import { maybeRedrawHUD, redrawHUD } from "./world/HUDRenderer.js";

export class Level {
  constructor(pkg, assets, opts = {}) {
    this.pkg = pkg;
    this.assets = assets;
    this.hudGfx = opts.hudGfx || null;
    this.events = opts.events || null;

    this.levelData = pkg.level;
```

Game.js

akshirsa_sidequest_W6

CHAT

```javascript
    this._unsubs.forEach((u) => u());
    this._unsubs = [];
  }

  // ------------------------
  // ani safety helpers
  // ------------------------

  _setAniSafe(sprite, name) {
    if (!sprite?.anis || !sprite.anis[name]) return false;
    sprite.ani = name;
    return true;
  }

  _setAniFrame0Safe(sprite, name) {
    if (!this._setAniSafe(sprite, name)) return false;
    if (sprite.ani) sprite.ani.frame = 0;
    return true;
  }

  // ------------------------
  // build / loop
  // ------------------------

  build() {
    world.gravity.y = this.GRAVITY;

    // Reset timer on build (new level instance or rebuild)
    this.elapsedMs = 0;

    // 1) Build tile groups + spawn from tilemap.
    // TileBuilder is responsible for creating:
    // - static tile groups (ground/platforms/walls)
    // - interactive groups (leaf/fire)
    // - boar group + tile spawning ('b') via Tiles()
    buildTilesAndGroups(this);

    // 2) Player entity + controller (WORLD)
    this.player = new PlayerEntity(this.pkg, this.assets);
    this.player.buildSprites();
    this.playerCtrl = new PlayerController(this.player, { events: this.events });

    // 3) Cache spawns + wire interactions (ONE TIME) + hook boar collisions
    this._cacheLeafSpawns();
    cacheBoarSpawns(this);

    this._wirePlayerInteractionsOnce(); // player<->leaf/fire
    this._wireBoarFireRuleOnce(); // boar<->fire (wired once to boar group)
    this._rebindPlayerBoarCollide(); // player<->boar (bind to current boar group)
```

Game.js

akshirsa_sidequest_W6

CHAT

```javascript
update({ input }) {
  const playerDead = this.player?.dead === true;

  // 0) WORLD timer (STOP on terminal states)
  // (Put this early so it counts time even if logic below early-outs later.)
  if (!this.won && !playerDead) {
    this.elapsedMs += deltaTime;
  }

  // 1) AI (freeze boars if dead/won, matching monolith feel)
  if (!playerDead && !this.won) {
    updateBoars(this);
  } else if (this.boar) {
    for (const e of this.boar) e.vel.x = 0;
  }

  // 2) Player consumes input snapshot
  this.playerCtrl.update({
    input,
    solids: this._solids(),
    bounds: this.bounds,
    won: this.won,
  });

  // 3) Optional safety checks (remove later if desired)
  this._preStepPhysicsSanity();

  // 4) Physics step
  world.step();

  // 5) World rules
  this._fallResetIfNeeded();

  // 6) HUD refresh (score/health only; timer not shown)
  maybeRedrawHUD(this);
}

drawWorld() {
  allSprites.draw();
}

restart() {
  this.won = false;
  this.score = 0;

  // reset timer on restart
  this.elapsedMs = 0;

  // reset player entity/controller state
```

Game.js

akshirsa_sidequest_W6

CHAT

```
// Player/boar contact damage must be attached to the CURRENT boar Group.
// Guard so we only bind once per boar-Group instance (prevents stacking across restarts)
_rebindPlayerBoarCollide() {
  const p = this.playerCtrl?.sprite;
  const g = this.boar;
  if (!p || !g) return;

  // If we already bound to this exact Group object, do nothing.
  if (this._boarGroupBoundForPlayer === g) return;
  this._boarGroupBoundForPlayer = g;

  p.collides(g, (playerSprite, boarSprite) => {
    if (boarSprite.dying || boarSprite.dead) return;
    this.playerCtrl.damageFromX(boarSprite.x);
  });
}

_rescueLeaf(playerSprite, leafSprite) {
  if (!leafSprite.active) return;

  leafSprite.active = false;
  leafSprite.visible = false;
  leafSprite.removeColliders();

  this.score++;
  this.events?.emit("leaf:collected", { score: this.score, winScore: this.WIN_SCORE });

  if (this.score >= this.WIN_SCORE) {
    this.won = true;

    // freeze player immediately (monolith behavior)
    playerSprite.vel.x = 0;
    playerSprite.vel.y = 0;

    this.events?.emit("level:won", { score: this.score, winScore: this.WIN_SCORE, elapsedM
  }
}

// Hook called from "player:attackWindow"
_tryHitBoar({ facing, x, y }) {
  if (!this.boar) return;
  if (this.player.attackHitThisSwing) return; // optional guard

  const rangeX = Number(this.tuning.player?.attackRangeX ?? 20);
  const rangeY = Number(this.tuning.player?.attackRangeY ?? 16);

  const playerFeetY = y + (this.playerCtrl.sprite?.h ?? 12) / 2;

  for (const e of this.boar) {
```

Game.js

akshirsa_sidequest_W6

CHAT

```javascript
_damageBoar(e, facingDir) {
  if (e.dead || e.dying) return;

  const flashFrames = Number(this.tuning.boar?.flashFrames ?? 5);
  const knockFrames = Number(this.tuning.boar?.knockFrames ?? 7);
  const knockX = Number(this.tuning.boar?.knockbackX ?? 1.2);
  const knockY = Number(this.tuning.boar?.knockbackY ?? 1.6);

  e.hp = Math.max(0, (e.hp ?? Number(this.tuning.boar?.hp ?? 3)) - 1);
  e.flashTimer = flashFrames;

  this.events?.emit("boar:damaged", { hp: e.hp, x: e.x, y: e.y });

  if (e.hp <= 0) {
    // Match monolith: enter "dying" first, then "dead" once grounded in BoarSystem.
    e.dying = true;
    e.vel.x = 0;

    // Ensure it can't collide while dying
    e.collider = "none";
    e.removeColliders();

    // In the monolith you set "throwPose" here (not "death").
    // Actual death animation starts later inside BoarSystem when grounded.
    this._setAniFrame0Safe(e, "throwPose");

    this.events?.emit("boar:died", { x: e.x, y: e.y });
    return;
  }

  // knockback
  e.knockTimer = knockFrames;
  e.vel.x = facingDir * knockX;
  e.vel.y = -knockY;

  this._setAniFrame0Safe(e, "throwPose");
}

// ------------------------
// helpers
// ------------------------

_cacheLeafSpawns() {
  this.leafSpawns = [];
  for (const s of this.leaf) {
    s.active = true;
    this.leafSpawns.push({ s, x: s.x, y: s.y });
  }
}
```

Game.js

akshirsa_sidequest_W6

CHAT

```
  _preStepPhysicsSanity() {
    for (const s of allSprites) {
      if (!s) continue;

      if (!Number.isFinite(s.x) || !Number.isFinite(s.y)) {
        console.warn("[SANITY] removing sprite with bad position:", { x: s.x, y: s.y });
        s.remove?.();
        continue;
      }

      // NOTE: In p5play v3, w/h may be getter-only and still valid to read.
      if ("w" in s && (!Number.isFinite(s.w) || s.w <= 0)) {
        console.warn("[SANITY] removing sprite with bad width:", { w: s.w, x: s.x, y: s.y }
        s.remove?.();
        continue;
      }

      if ("h" in s && (!Number.isFinite(s.h) || s.h <= 0)) {
        console.warn("[SANITY] removing sprite with bad height:", { h: s.h, x: s.x, y: s.y
        s.remove?.();
        continue;
      }

      if (s.body) {
        const p = s.body.getPosition?.();
        if (!p || !Number.isFinite(p.x) || !Number.isFinite(p.y)) {
          console.warn("[SANITY] removing sprite with bad body position:", { p });
          s.remove?.();
        }
      }
    }
  }

  // ------------------------
  // HUD API (delegates)
  // ------------------------
  redrawHUD() {
    redrawHUD(this);
  }
}
```

For adding the new level, refer to my previous response: update levels.json with a second level (e.g., "ex5_level2") that has the same view (including background) but a different tilemap array with repositioned tiles. Then modify main.js to have `LEVEL_IDS = ["ex5_level1", "ex5_level2"]` and progress on win, and update Game.js to reset properly. No changes needed to Level.js itself. Test by winning the first level—it should load the second automatically. If issues, check the console for JSON or loading errors.

Game.js